# CS 214 Fall 2025
# Project II: Spelling checker

David Menendez

Due: October TBD, at 11:59 PM (ET)

For this assignment, you and your partner will write a program that detects and reports spelling errors in one or more text files. This project will give you practice using the Posix functions for reading and writing files and for traversing directories.

## 1 Program

Write a program `spell` that reads a dictionary file and checks the words in one or more text files to see if any are not listed in the dictionary. For each word not found in the dictionary, `spell` will report the file name, line number, and column number of the unknown word.

### 1.1 Interface

In order to operate, users must specify (1) a dictionary file (word list), (2) one or more input files, and (3) a file suffix to use when scanning directories.

The general form is:

```
spell [-s {suffix}] {dictionary} [{file or directory}]*
```

The dictionary and input sources are given by the required arguments. The file suffix is given by an optional argument.

**Required arguments** The first non-optional argument will be a path to a *dictionary file* (see section 1.2). This argument must be present.

Subsequent non-optional arguments will be paths to normal files (section 1.3) or directories (section 1.4). If there are no additional arguments, `spell` will read text from standard input.

**Optional argument** If an argument begins with `-s`, then the next argument will specify the file name suffix to be used when scanning directories. It is an error for `-s` to occur as the last argument. If multiple `-s` arguments are present, you may report an error or use the suffix given by the last argument.

For simplicitly, you may forbid `-s` from occuring later than the first argument.

If the optional argument is not present, `spell` will use the default value for the file name suffix, ".txt".

**Examples** Using dictionary `dict` and checking text from standard input:

```
spell dict
```

Using dictionary `words` and checking text from files `foo`, `bar`, and `baz`:

```
spell words foo bar baz
```

Using dictionary `dict.txt` and reading files ending with `.txt` from directory `quux` and its subdirectories, recursively:

```
spell dict.txt quux
```

Using dictionary `asdf` and reading `foo` and also files ending with `.bar` in the directories `quux/bim` and `bom` (recursively):

```
spell -s .bar asdf quux/bim bom
```

## 1.2 Dictionary file

A dictionary file contains one or more "correct" words, one per line. The words may include duplicates and are not required to be in any particular order.

## 1.3 Text file

For our purposes, a text file contains bytes encoding characters using the ASCII encoding. A text file can be broken into *lines*, which are a sequence of characters ending with a newline character. These lines may contain *words*, which we will consider a sequence of non-whitespace characters.

We number all the lines in a file, starting from one. Each character in a line has a *column number*, which also starts from one. We will want to track the line and column number for each word, defined as the column number of the word's first character.

**Digits** Words consisting entirely of digits and other non-letter symbols are ignored. For example, "42", "$100", and "1,2023!84-39" will be skipped, but "23-skidoo", "super8", and "i18n" would be checked.

**Punctuation and symbols** Words consisting entirely of non-letter characters are skipped. If a word ends in one or more symbol characters (that is, not whitespace, letters, or digits), those trailing characters are ignored.

If a word begins with one or more opening parentheses, brackets, braces, apostrophes, or quotation marks, they will be ignored.

If a dictionary contains "foo", then "foo!", "foo@-", and "(foo?)" are accepted.

Punctuation and symbols occuring mid-word are never ignored.

**Capitalization** Letters that are capitalized in the dictionary must be capitalized in words, but lowercase letters in the dictionary will match capital or lower-case letters.

If the dictionary contains "foo" and "Bar", then "Foo", "fOO", "BAr" and "BAR" are accepted, but not "bar".

A recommended approach is to use a *normalized* form of the word as a search key (e.g., by lower-casing all letters) and then checking whether the capitalization in the input file is an acceptable form of the dictionary entry.

### 1.4 Directories

When `spell` is given a directory name as an argument, it will perform a recursive directory traversal to find files to read. Files and directories whose names begin with a period are skipped and not considered. Any other directories will be scanned. Files whose names end with the *file name suffix* will be checked for spelling errors.

Unless specified by the user, the file name suffix is ".txt".

We do not require `spell` to check the files in any particular order, but all specified files must be checked.

(Note that the requirements to ignore files beginning with a period and not ending with ".txt" only apply to directory traversal, not to files in the argument list.)

### 1.5 Reporting errors

Each time `spell` finds an incorrect word, it will report the word along with the file containing the word and the line and column number.

For example:

```
$ spell my_dict file1 my_files
file1:35:8 foom
my_files/bad.txt:1:1 Badd
my_files/baz/bar.txt:8:19 almost-correkt
```

If only one file name (not directory) is specified, or if reading from standard input, you may omit the file name.

Print an error message if a file cannot be opened.

`spell` should exit with status `EXIT_SUCCESS` if all files could be opened and contained no incorrect words, and `EXIT_FAILURE` otherwise.

## 2 Coding

We can break `spell` into three major components:

1. Finding and opening all the specified files, including directory traversal
2. Reading the file and generating a sequence of position-annotated words
3. Checking whether a word is contained in the dictionary

You are encouraged to design your code so that these components can be written and tested separately.

You should devise a data structure to represent the dictionary. You will need to do a lookup for each word in the text documents being checked, so choose a data structure that permits quick lookups. As the dictionary is not modified once created, an array with binary search can be effective, but you are free to use trees, hash-tables, or even more powerful structures such as tries. Ideally, your code will have acceptable performance even when using the large dictionaries containing hundreds of thousands of words.

You *must* use `read()` to read the dictionary and text files. No other file reading functions are permitted.

Use of `write()` for printing error messages is suggested, but not required.

# 3 Testing

Your testing plan should be designed to catch bugs, but also to give confidence that your program does not contain bugs. Consider the requirements for `spell` and use these to design scenarios where `spell` should exhibit some known behavior.

Describe your test scenarios in your README, including which problems they are intended to detect.

# 4 Submission

Submit your source code, Makefile, AUTHOR, and README in a Tar archive, along with any test files and dictionaries you mention in your README. The root of the archive should be a directory called P2 (case-sensitive).

Ideally, we should be able to extract your archive, move into P2, call make, and begin executing `spell`. That is,

```
$ tar -xf your-p2.tar
$ cd P2
$ make
$ ./spell ../dict ../testfile
```

**AUTHOR** In your AUTHOR file, include only the NetIDs for both partners.

**README** In your README, note the names and NetIDs for both partners. Describe any notes on your design, or limitations if you were not able to implement everything. Also describe your requirements, test scenarios, and test cases.

# 5 Grading

Grading will be based on

- Correctness: whether your library operates as intended
- Design: the clarity and robustness of your code, including modularity, error checking, and documentation
- The thoroughness and quality of your test plan